Performance study of kernel TLS handshakes

Alexander Krizhanovsky, Ivan Koveshnikov

Tempesta Technologies, Inc.

ak@tempesta-tech.com, ik@tempesta-tech.com

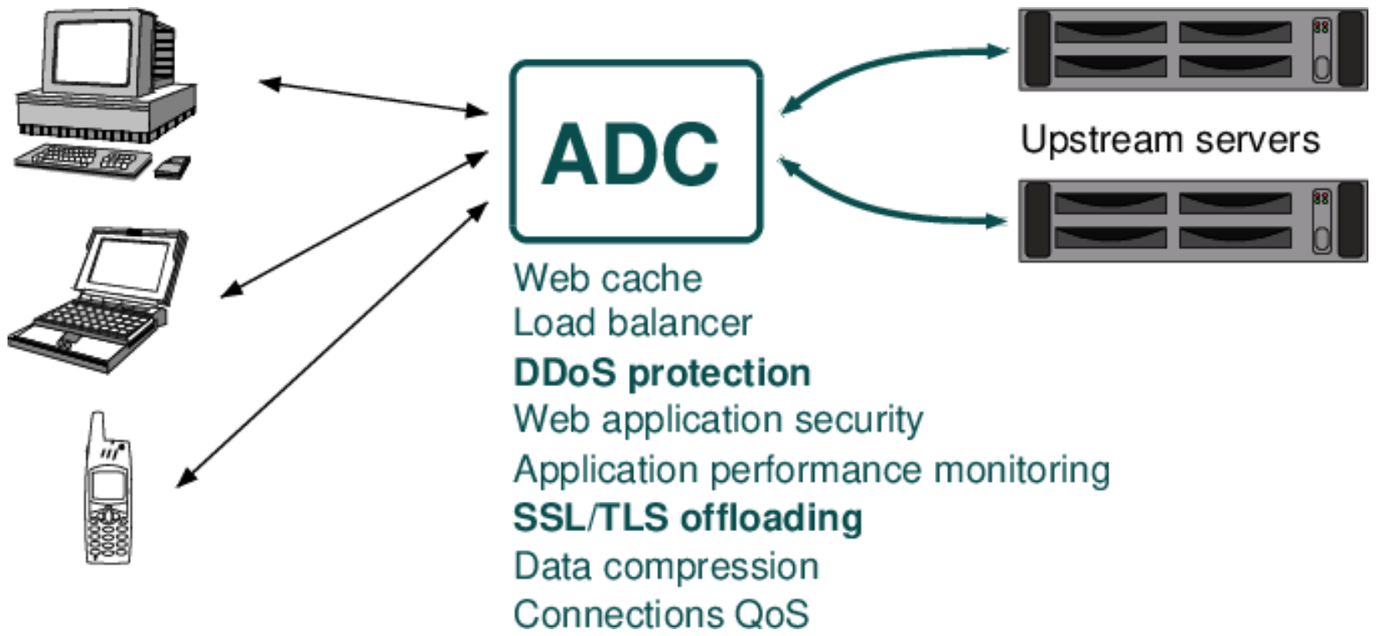
Agenda

- Why kernel TLS handshakes?
- Benchmarks
- Current state of Tempesta TLS (part 3)
  - Part 2, “Kernel TLS handshakes for HTTPS DDoS mitigation”, Netdev 0x12, https://netdevconf.info/0x12/session.html?kernel-tls-handhakes-for-https-ddos-mitigation
  - Part 1, “Kernel HTTP/TCP/IP stack for HTTP DDoS mitigation”, Netdev 2.1, https://netdevconf.info/2.1/session.html?krizhanovsky
- Design proposal for the Linux mainstream

Tempesta TLS

https://netdevconf.info/0x12/session.html?kernel-tls-handhakes-for-https-ddos-mitigation

- Part of **Tempesta FW**, an open source *Application Delivery Controller*
- Open source alternative to **F5 BIG-IP** or **Fortinet ADC**
- TLS in ADCs:
  - “TLS CPS/TPS” is a common specification for network security appliances
  - BIG-IP **30-50%** faster than Nginx/DPDK in VM/CPU workload
    https://www.youtube.com/watch?v=Plv87h8GtLc



Tempesta Technologies

TLS handshake issues

- DDoS on TLS handshakes (asymmetric DDoS)
- Privileged address space for sensitive security data
  - Varnish: TLS is processed in separate process Hitch
    http://varnish-cache.org/docs/trunk/phk/ssl.html
  - Resistance against attacks like CloudBleed
    https://blog.cloudflare.com/incident-report-on-memory-leak-caused-by-cloudflare-parser-bug/
- A light-weight kernel implementation can be faster
  - ...even for session resumption
  - There are modern research in the field

TLS libraries performance issues

- Copies, memory initializations/erasing, memory comparisons
- `memcpy()`, `memset()`, `memcmp()` and their constant-time analogs
- Many dynamic allocations
- Large data structures
- Some math is outdated

```
12.79%  libc-2.24.so          _int_malloc
 9.34%  nginx(openssl)        __ecp_nistz256_mul_montx
 7.40%  nginx(openssl)        __ecp_nistz256_sqr_montx
 3.54%  nginx(openssl)        sha256_block_data_order_avx2
 2.87%  nginx(openssl)        ecp_nistz256_avx2_gather_w7
 2.79%  libc-2.24.so          _int_free
 2.49%  libc-2.24.so          malloc_consolidate
 2.30%  nginx(openssl)        OPENSSL_cleanse
 1.82%  libc-2.24.so          malloc
 1.57%  [kernel.kallsyms]     do_syscall_64
 1.45%  libc-2.24.so          free
 1.32%  nginx(openssl)        ecp_nistz256_ord_sqr_montx
 1.18%  nginx(openssl)        ecp_nistz256_point_doublex
 1.12%  nginx(openssl)        __ecp_nistz256_sub_fromx
 0.93%  libc-2.24.so          __memmove_avx_unaligned_erms
 0.81%  nginx(openssl)        __ecp_nistz256_mul_by_2x
 0.75%  libc-2.24.so          __memset_avx2_unaligned_erms
 0.57%  nginx(openssl)        aesni_ecb_encrypt
 0.54%  nginx(openssl)        ecp_nistz256_point_addx
 0.54%  nginx(openssl)        EVP_MD_CTX_reset
 0.50%  [kernel.kallsyms]     entry_SYSCALL_64
```

Performance tools & environment

- **`tls-perf`** (*https://github.com/tempesta-tech/tls-perf*)
  - *establish & drop many TLS connections in parallel*
  - *like TLS-THC-DOS, but faster, more flexible, more options*
- **`wrk`** (*https://github.com/wg/wrk*)
  - *full HTTP**S** transaction w/ new TLS connection or session resumption*
- OpenSSL/Nginx as the reference (NIST secp256r1, ECDHE & ECDSA)
- Environment
  - hardware server w/ loopback connection
  - hardware servers w/ 10Gbps NIC (ping time < 0.06ms)
  - host to KVM (no vAPIC)

Tempesta Technologies

The source code

https://github.com/tempesta-tech/tempesta/tree/master/tls

- Still in-progress: we implement some of the algorithms on our own
- Initially the fork of **mbed TLS** 2.8.0 (*https://tls.mbed.org/*)
  - very portable and easy to move into the kernel
  - cutting edge security
  - too many memory allocations (https://github.com/tempesta-tech/tempesta/issues/614)
  - big integer abstractions (*https://github.com/tempesta-tech/tempesta/issues/1064*)
  - inefficient algorithms, no architecture-specific implementations, ...
- We also take parts from **WolfSSL** (*https://github.com/wolfSSL/wolfssl/*)
  - very fast, but not portable
  - security *https://github.com/wolfSSL/wolfssl/issues/3184*

Tempesta Technologies

Performance improvements from mbedTLS

- Original mbed TLS 2.8.0 ported into the kernel (1CPU VM)

```
HANDSHAKES/sec:   MAX 193; AVG 51; 95P 0; MIN 2
LATENCY (ms):     MIN 352; AVG 2566; 95P 4001; MAX 6949
```

- Current code (1CPU VM): **x40** better performance!

```
HANDSHAKES/sec:   MAX 2761; AVG 2180; 95P 1943; MIN 1844
LATENCY (ms):     MIN 134; AVG 307; 95P 757; MAX 2941
```

Tempesta Technologies

TLS 1.2 full handshakes: Tempesta vs Nginx/OpenSSL

- Nginx 1.14.2, OpenSSL 1.1.1d (1CPU VM)

```
HANDSHAKES/sec:   MAX 2410; AVG 1545; 95P 1135; MIN 1101
LATENCY (ms):     MIN 188; AVG 1340; 95P 1643; MAX 2154
```

- Tempesta FW (1CPU VM): **40%** better performance, **x4** lower latency

```
HANDSHAKES/sec:   MAX 2761; AVG 2180; 95P 1943; MIN 1844
LATENCY (ms):     MIN 134; AVG 307; 95P 757; MAX 2941
```

Tempesta Technologies

TLS 1.2 session resumption: Tempesta vs Nginx/OpenSSL

- *Nginx 1.19.1, OpenSSL 1.1.1d (2CPU Baremetal)*

```
HANDSHAKES/sec:   MAX 20098; AVG 19203; 95P 17262; MIN 8739
LATENCY (ms):     MIN 10; AVG 74; 95P 246; MAX 925
```

- Tempesta FW (2CPU Baremetal): **80%** better performance, same latency

```
HANDSHAKES/sec:   MAX 37575; AVG 35263; 95P 31924; MIN 8507
LATENCY (ms):     MIN 9; AVG 77; 95P 268; MAX 28262
```

- We noticed large latency gaps for a just few connections during benchmarks, but the rest has the same latency.
  https://github.com/tempesta-tech/tempesta/issues/1434

Tempesta Technologies

TLS 1.2 session resumption: Nginx/OpenSSL on different kernels

- Nginx 1.19.1, OpenSSL 1.1.1d, Linux 5.7 (2CPU Baremetal)

```
HANDSHAKES/sec:  MAX 18233; AVG 17274; 95P 15892; MIN 8329
LATENCY (ms):    MIN 10; AVG 70; 95P 102; MAX 853
```

- Nginx 1.19.1, OpenSSL 1.1.1d, Linux 4.14 (2CPU Baremetal)

```
HANDSHAKES/sec:  MAX 20098; AVG 19203; 95P 17262; MIN 8739
LATENCY (ms):    MIN 10; AVG 74; 95P 246; MAX 925
```

- Meltdown&Co mitigations cost more with network I/O Increase
  https://www.phoronix.com/scan.php?page=article&item=spectre-meltdown-2&num=10

Tempesta Technologies

Assumption about KPTI

- TLS handshakes system calls:
  - network I/O (or 0-rtt)
  - memory allocations (or cached)
  - system random generator (or `RDRAND`)
- So we though KPTI will hurt TLS handshakes…
  https://mariadb.org/myisam-table-scan-performance-kpti/
  https://www.phoronix.com/scan.php?page=article&item=spectre-meltdown-2&num=10

TLS	w/o KPTI	w/ KPTI	Delta
1.2	8735	8401	-3.82%
1.3	7734	7473	-3.37%

Tempesta Technologies

TLS handshakes in full HTTPS transaction

- Nginx 1.19.1, OpenSSL 1.1.1d (2CPU Baremetal)

	Handshake only, HS/s	1Kb response, RPS	10Kb response, RPS
TLS 1.2 New Sessions	7225	6402	6238
TLS 1.2 Resume Sessions	17274	13472	12630

- *Handshakes determine performance for short living connections*

Tempesta Technologies

ECDSA & ECDHE mathematics: Tempesta, OpenSSL, WolfSSL

- OpenSSL 1.1.1d

```
256 bits ecdsa (nistp256)      36472.5 sign/s
256 bits ecdh (nistp256)       16619.8 op/s
```

- WolfSSL (current master)

```
ECDSA    256 sign                41527.239 ops/sec
ECDHE    256 agree               55548.005 ops/sec    (?)
```

- Tempesta TLS *(unfair)*

```
ECDSA sign (nistp256):  ops/s=27261
ECDHE srv (nistp256):   ops/s=6690
```

- We need more work…
- ...but OpenSSL & WolfSSL don’t include **ephemeral keys** generation

Why faster?

- No memory allocations in run time
- No context switches
- No copies on socket I/O
- Less message queues
- Zero-copy handshakes state machine
  https://netdevconf.info/0x12/session.html?kernel-tls-handhakes-for-https-ddos-mitigation

Elliptic curves (asymmetric keys): the big picture

OpenSSL: *“Fast prime field elliptic-curve cryptography with 256-bit primes"* by Gueron and Krasnov, *2014*.

- `Q = m * P` - the most expensive elliptic curve operation

```
for i in bits(m):
    Q ← point_double(Q)
    if m_i == 1:
        Q ← point_add(Q, P)
```

- Point multiplications in TLS handshake:
  - fixed `m`: ECDSA signing: fixed point multiplication
  - **unknown** `m`: ECDHE key pair generation **and** shared secret generation
  - Scalar `m` is always security sensitive

Tempesta Technologies

The math layers

- Different multiplication algorithms for fixed and unknown point
  - *”Efficient Fixed Base Exponentiation and Scalar Multiplication based on a Multiplicative Splitting Exponent Recoding”, Robert et al 2019*
- Point doubling and addition - everyone seems use the same algorithms
- Jacobian coordinates: different **modular inversion** algorithms
  - *“Fast constant-time gcd computation and modular inversion”, Bernstein et al, 2019*
- **Modular reduction** for scalar multiplications:
  - Montgomery has overhead vs FIPS speed: if we use less multiplcations it could make sense to use different reduction method FIPS *(seems deadend)*
  - *“Low-Latency Elliptic Curve Scalar Multiplication” Bos, 2012*

=> *Balance between all the layers*

Tempesta Technologies

Side channel attacks (SCA) resistance

- Timing attacks, simple power analysis, differential power analysis etc.
- Protections against SCAs:
  - Constant time algorithms
  - Dummy operations
  - Point randomization
  - *e.g. modular inversion **741 vs 266** iterations*
- **RDRAND** allows to write faster non-constant time algorithms
  - SRBDS mitigation costs about **97% performance**

    https://software.intel.com/security-software-guidance/insights/processors-affected-special-register-buffer-data-sampling?fbclid=IwAR1ifj3ZuAtNOabKkj3vFItBLSvOnMqlxH2I-QeN5KB-aji54J1BCJa9lLk

    https://www.phoronix.com/scan.php?page=news_item&px=RdRand-3-Percent&fbclid=IwAR2vmmR_LiroekUuw7KMRaHB7KThpqz0tIr1fX2GCW3HAvwt5Kb1p9xpLKo

Memory usage & SCA

- ex. ECDSA precomputed table for fixed point multiplication
  - mbed TLS: *~8KB* dynamically precomputed table, point randomization, constant-time algorithm, *full table scan*
  - OpenSSL: *~150KB static table, full scan*
  - *WolfSSL: ~150KB, direct index access*
    https://github.com/wolfSSL/wolfssl/issues/3184

=> 150KB is far larger than L1d cache size, so many cache misses

Big Integers (aka MPIs)

"BigNum Math: Implementing Cryptographic Multiple Precision Arithmetic", by Tom St Denis

- All the libraries use them (*not in hot paths*), mbed TLS **overuses** them
- `linux/lib/mpi/`, `linux/include/linux/mpi.h`

```
typedef unsigned long int mpi_limb_t;

struct gcry_mpi {
    int alloced;        /* array size (# of allocated limbs) */
    int nlimbs;         /* number of valid limbs */
    int nbits;          /* the real number of valid bits (info only) */
    int sign;           /* indicates a negative number */
    unsigned flags;
    mpi_limb_t *d;      /* array with the limbs */
};
```

- Need to manage variable-size integers

Tempesta Technologies

Memory profiles

- The MPI fields must be properly initialized
- Each handshake involves hundreds of temporary MPIs
- Memory profile
  - **preallocate** a profile for each curve and ciphersuite on start
  - Placed in contiguous memory pages
  - Each new handshake just **copies** (*streamed*) the whole profile
  - At the end the whole profile is **zeroed** (*streamed*) and **freed**

Tempesta Technologies

TLS socket API proposal

```
key_serial_t sni_kr = add_key("keyring", "example.com", NULL, 0, 0);
pub_key = add_key("tls_cert", "tlscrt:", cert, cert_len, sni_kr);
priv_key = add_key("tls_priv", "tlspk:", priv_key, pk_len, sni_kr);

sd = socket(AF_INET6, SOCK_STREAM, 0); bind(sd, …);

struct tls_hs_crypto_desc ci = {
    .version = TLS_1_2_VERSION | TLS_1_3_VERSION,
    .cipher_suite = ECDHE-ECDSA-AES128-GCM-SHA256,
    .tls_keyring = tls_kr,
};
setsockopt(sd, SOL_TCP, TCP_ULP, "tls", sizeof("tls"));
setsockopt(sd, SOL_TLS, TLS_HS, &ci, sizeof(ci));

listen(sd, …); new_sd = accept(sd, …);
read(new_sd, decrypted_data, …);
```

- More details and the discsussion on
  https://github.com/tempesta-tech/tempesta/issues/1433

Tempesta Technologies

Network processing

- Server-side only
- The full TLS handshake is in **softirq** (just like TCP)
- Fallback to a user-space TLS library on `ClientHello`
- Batches of handshakes in **1 FPU context**

FPU save/restore price

- `kernel_fpu_begin()`/`kernel_fpu_end()` per crypto call is expensive
- Tempest FW (re-)stores FPU context on start/exit softirq handler

https://github.com/tempesta-tech/blog/blob/master/kstrings/memcpy_res.kernel

Raw:

```
unaligned:     304ms
        8:     110ms
       20:     153ms
       64:     145ms
      120:     181ms
      256:     210ms
      320:     237ms
      512:     309ms
      850:     464ms
     1500:     358ms
```

Safe:

```
unaligned:     1591ms
        8:     2865ms
       20:     2887ms
       64:     2887ms
      120:     2864ms
      256:     3011ms
      320:     3034ms
      512:     3113ms
      850:     3278ms
     1500:     1710ms
```

How big is the code?

- Already in the kernel
  - asymmetric keys management (seems must be extended)
  - curve25519, RSA (seems must be extended)
  - all the symmetric crypto
- The new code

```
$ wc -l bignum_x86-64.S bignum_asm.h ciphersuites.* crypto.* ecdh.* ec_p256.c ecp.*
mpool.* pk.* tls_internal.h tls_srv.c tls_ticket.* ttls.*
  1394 bignum_x86-64.S          315 ecp.c                    1653 ec_p256.c
    65 bignum_asm.h             271 ecp.h                     430 tls_ticket.c
   276 ciphersuites.c           425 mpool.c                   114 tls_ticket.h
   106 ciphersuites.h            41 mpool.h                  2783 ttls.c
   480 crypto.c                 411 pk.c                      611 ttls.h
   226 crypto.h                 172 pk.h
   225 ecdh.c                   425 tls_internal.h
    76 ecdh.h                  2202 tls_srv.c

 12701 total
```

Tempesta Technologies

TODO

- More NIST p256 curve optimizations

  (https://github.com/tempesta-tech/tempesta/issues/1064)

- TLS 1.3

  (https://github.com/tempesta-tech/tempesta/issues/1031)

- Moving to the kernel assymetric keys API

  (https://github.com/tempesta-tech/tempesta/issues/1332)

Tempesta Technologies

Thanks!

- We’d love to know
  - If you can benefit from the kernel TLS handshakes
  - feedback on API, implementation and so on
  - **your questions?**

https://github.com/tempesta-tech/tempesta

ak@tempesta-tech.com *ik@tempesta-tech.com*

Tempesta Technologies